

A Pattern-oriented Design Framework for Self-adaptive Software Systems

Paolo Arcaini
National Institute of Informatics
Japan
arcaini@nii.ac.jp

Raffaella Mirandola
Politecnico di Milano
Italy
raffaella.mirandola@polimi.it

Elvinia Riccobene
Università degli Studi
di Milano, Italy
elvinia.riccobene@unimi.it

Patrizia Scandurra
Università degli Studi
di Bergamo, Italy
patrizia.scandurra@unibg.it

Abstract—Multiple interacting MAPE-K loops, structured according to specific interaction patterns, have been introduced to design the adaptation logic in case of decentralized self-adaptive software systems. Designing such complex systems requires the availability of tools where MAPE patterns can be easily instantiated to provide fast architectural solutions, and the encoding towards specific domains is facilitated by automatic mapping of such pattern instantiations in domain-specific languages; validation and verification must be also supported to assure correct development of reliable systems.

In this paper, we present a pattern-oriented framework, based on the MSL (MAPE Specification Language) modeling language, for the design of self-adaptive systems. The framework supports: (i) explicit modeling of the adaptation logic in terms of patterns of interactive MAPE-K loops; (ii) ability to tailor MSL models for a specific application domain and synthesize from them other modeling artifacts/code according to a target implementation context and scope (e.g., OpenHAB); (iii) ability to perform validation and verification of MSL models by means of the ASMETA formal framework.

Keywords—Pattern-oriented design; self-adaptation; MAPE-K loops; MAPE patterns; ASMETA; OpenHAB

I. INTRODUCTION

Self-adaptation (SA) is considered an essential characteristic of software systems that operate in dynamic environments and deal with highly changing operational conditions. Several architectural solutions adopted to realize self-adaptation are centered around feedback control loops that monitor and adapt managed parts of a software system following a MAPE-K (Monitor, Analyze, Plan, and Execute over a shared Knowledge) paradigm [1]. To deal with decentralized systems, multiple interacting MAPE-K loops, structured according to specific design patterns, have been introduced [2] to be able to model different interacting ways among the control loops.

Following the design up-front methodology, we pursue an engineering approach contributing to a rigorous, but user-friendly, development of self-adaptive systems. Such an approach requires tools where MAPE patterns can be

easily instantiated and included in system architectures to provide fast architectural solutions. Besides, to foster a wide use and evaluation, these tools should include both encoding towards specific domains provided by automatic mapping of such pattern instantiations into domain-specific languages, and the ability to perform V&V (validation and verification) to assure correct development of reliable systems.

In this paper we present a pattern-oriented framework based on a modeling language, called MSL (MAPE Specification Language) [3], for the design of self-adaptive systems. MSL adopts the same modeling concepts of the MAPE patterns as in [2]. Within the framework, MSL models can be automatically encoded in OpenHAB control rules, so providing fast implementation of architectural solutions in the context of home automation. This is an example of one of the possible domain-specific implementations of MSL models. To empower the framework with the ability to perform V&V analysis, *self-adaptive Abstract State Machines* (self-adaptive ASMs) [4] formal models can be generated from MSL models. These ASM models express and guarantee by construction the loops interaction semantics and can be simulated, refined, and verified to provide early feedback about the correct loops execution as devised in [4].

To summarize, the proposed framework supports the (i) explicit modeling of the adaptation logic in terms of patterns of interactive MAPE-K loops; (ii) ability to tailor such models for a specific application domain and synthesize from them other modeling artifacts or code according to a target implementation context and scope (e.g., OpenHAB); (iii) ability to perform V&V of MSL models by means of the ASMETA (ASM mETAmodeling) [5] formal framework.

Related work Hereafter, we give an overview about existing tools allowing the explicit modeling of feedback loops for SA. SOTA (State Of The Affairs) [6], supported by the Eclipse plug-in SimSOTA, is a goal-oriented approach for modeling, simulating and validating self-adaptive systems. Unlike our approach, SOTA adopts a semi-formal notation, namely UML activity diagrams, to model the behavior of feedback loops. ACTRESS [7] is an Eclipse-based framework for modeling, automated analysis, and Java-like code synthesis of self-adaptive systems. The framework is centered around an actor-oriented DSL for defining feedback control loops; however, patterns are not supported. Activ-

P. Arcaini is supported by ERATO HASUO Metamathematics for Systems Design Project (No. JPMJER1603), JST. Funding Reference number: 10.13039/501100009024 ERATO. E. Riccobene has received funding from the European Union Horizon 2020 research and innovation programme under the Marie Skłodowska-Curie grant agreement No 691178. R. Mirandola has been supported by KK-Stiftelsen project No. 20170232 during her visiting period in Växjö, Sweden.

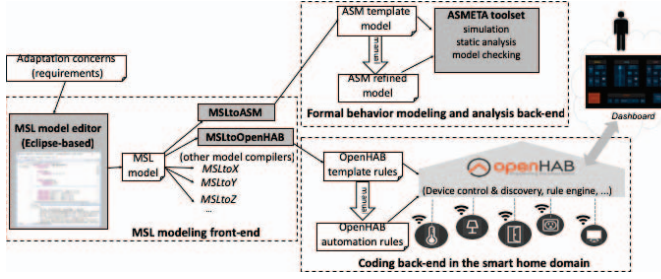


Figure 1. Overview of the MSL framework

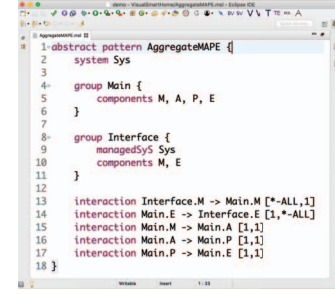


Figure 2. Aggregate MAPE pattern definition in the MSL editor

FORMS (Active FORMal Models for Self-adaptation) [8] is a virtual machine to realize self-adaptation with guarantee. Timed automata are used to model the MAPE-K loops accomplishing given adaptation goals. These MAPE-K loop models are verified off-line and then deployed directly on the virtual machine. ActivFORMS shares with us a formal description of the adaptation for verification purposes, but it avoids coding of the MAPE-K loop models in a target implementation language, since the models are translated directly in an internal representation (a state transition graph with task graphs) of the virtual machine. CyPhEF (CYber-PHYsical dEvelopment Framework) [9] supports a graphical notation for modeling the control architecture of a cyber-physical system by MAPE loop patterns.

The paper is structured as follows. Sect. II introduces the MSL framework, Sect. III its application to a case study, and Sect. IV concludes the paper.

II. THE MSL FRAMEWORK

The aim of the MSL framework [10] is to have, at the early design stage, a lightweight formalism for representing MAPE patterns and their instances as first-class citizens. These abstract models need then to be tailored and completed for the concrete application in a target implementation platform. Fig. 1 shows an overview of the MSL framework for modeling the adaptation layer of a self-adaptive system using MAPE patterns. The framework relies on a *modeling front-end* for editing MSL models on top of more specific modeling/coding and/or analysis *back-end* frameworks.

The MSL language allows the definition and instantiation of MAPE patterns in an expressive and concise manner using a textual notation; it adopts the same modeling concepts of the MAPE graphical notation presented in [2] to express the structure of MAPE components interactions, and, in addition, it provides constructs to fix some semantic variations in component interactions [3]. As an example, Fig. 2 shows the definition of the pattern *Aggregate MAPE* in the MSL model editor, as a set of abstract groups of MAPE components, managed subsystem, and components interactions with multiplicities. All the patterns proposed in [2] are expressed in MSL and available as built-in patterns.

MSL has been developed using the *grammarware* approach of Xtext, which, combined with the *modelware* approach of the Eclipse Modeling Framework (EMF), allows the automatic generation of a model editor (with content assist), a parser, and a basic validator (extended with custom validation rules for checking consistency between abstract and concrete patterns), and facilitates the development of generators toward other (back-end) frameworks.

From a simple definition in MSL of a structure of interactive MAPE loops, the corresponding representation in a target back-end framework can be obtained automatically using model compilers and then tailored/refined according to the target scope. Currently, two back-end frameworks are supported: the ASMETA¹ [5] toolset, to carry out formal analysis of MAPE-K loop models, and the rule engine of the *Open Home Automation Bus* (OpenHAB)², which is an open source runtime platform for home automation, to synthesize control home automation processes from MAPE loop models of smart home gateways. Other back-ends (such as those for modeling in UML or in other ADLs or other formal methods) could be supported as well, to complement and complete the design started in MSL.

ASMETA adopts a formal executable description of MAPE loops in terms of *self-adaptive Abstract State Machines* (self-adaptive ASMs) [4], and allows basic validation in terms of simulation (either guided or automatized), and verification in terms of static analysis and model checking. Specifically, the resulting ASM models (possibly further refined to specify the internal behavior of MAPE components) can be used, as devised in [4], for early prototyping of MAPE loop controllers and for verifying properties of distributed MAPE loops such as non-conflict, minimality, as well as correct interactions.

Typical and complex control home automation processes of smart home gateways can be designed in terms of structured MAPE-K feedback loops in MSL. The MAPE components of a MAPE-K feedback loop in MSL can be then mapped automatically into *Event-Condition-Actions* rules of the OpenHAB automation rule engine. These rules

¹<http://asmeta.sourceforge.net/>

²<https://www.openhab.org/>

share a common execution context (i.e., they can access and exchange variables with each other) and trigger each other according to the underlying execution order of the feedback loop and the component interactions as established by the MAPE pattern. The *Action* block of these rules can be then further refined to add application-specific control logic (any kind of task, e.g., “send a command to home devices”, “do mathematical calculations”, “start timers”, etc.). The automation evolution (including the devices’ state) for an OpenHAB instance configured for a specific smart home can be observed at runtime via the OpenHAB administration console or via a graphical dashboard accessible from a web browser (even remotely from a smartphone/tablet). This provides quickly a prototype of the managing subsystem (embedding the adaptation/control logic) over a managed one (the managed home) that can be independently designed and deployed. To this last purpose, OpenHAB abstracts from the underlying devices protocols and adopts a *Thing component model*³ to represent and access the home environment devices via software components and APIs.

MAPE loops for additional adaptation goals can be introduced at any moment: they can be modelled in MSL, refined and analyzed in ASMETA, and then coded, refined and deployed in OpenHAB at run-time.

III. SMART HOME AUTOMATION EXAMPLE

We illustrate the MSL framework with a smart home automation example. The adaptation logic of the smart home gateway that deals with complex automation control concerns can be designed, verified, and coded in the MSL framework in terms of MAPE-K feedback loops. Specifically, we illustrate the adaptation concern – *Comfortable heating* (CH) – of how to guarantee that the home’s heaters are at sufficient setting for comfort in all rooms, regardless of the room where the thermostat(s) are situated (remote temperature sensors) and regardless of sources of uncertainties in the home appliances/devices such as variability of heating sensors’ measures actually available and sudden changes in the home environment. Fig. 3 shows a concretization, called *ComfortableHeatingMAPE*, of the pattern *AggregateMAPE* for the adaptation concern CH. Essentially, there is a concrete MAPE group, *IntTemp*, that is responsible for monitoring rooms temperature via sensors and managing the heating thermostats setting accordingly. So it plays the role of *Interface* w.r.t. the heating system by providing both a component M and a component E. The group *MainCH* has a main sequential MAPE loop. It is responsible for aggregating temperature data from all rooms (through the component M of groups *IntTemp*), and then decides when to regulate or to turn on/off the heating equipment (driving the components E of the groups *IntTemp*) based on the home’s energy profile to guarantee

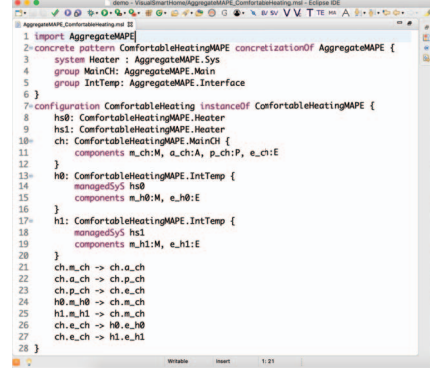


Figure 3. Aggregate MAPE pattern instance in the MSL editor

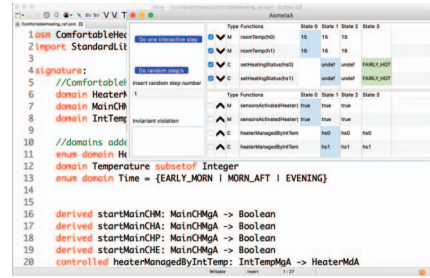


Figure 4. Example of animated simulation of a MAPE loop in ASMETA

comfort at any time. To complete the pattern instantiation, a configuration of the concrete pattern is also given in the second part of Fig. 3, as introduced by the keyword **configuration**: there are two managed heating systems (hs0 and hs1), and their interface MAPE groups (h0 and h1) interacting with one main MAPE group (ch).

The MAPE loop model in MSL can be then transformed automatically into a formal executable description in self-adaptive multi-agent ASMs (one agent per MAPE group) and the toolset ASMETA can be used for early behavior specification, simulation, and formal verification of the loop. Indeed, a generated ASM model is already executable as coordination schema of MAPE components (whose behavior is captured by means of rules of the ASM agent corresponding to the group), but with an empty implementation for the internal components behavior; therefore, the designer has to refine the ASM model by specifying the MAPE components behavior (ASM rules’ bodies). On the refined ASM model, the designer can perform some validation and verification activities in order to check that the designed MAPE loop behaves as expected for the considered adaptation concern. For example, the ASM model can be simulated and animated graphically in ASMETA to observe a MAPE loop execution, as shown in Fig. 4 for the loop CH. In the initial state State 0, the user interacts with the simulator through GUI dialogs (not shown in the Figure) to insert new values for the monitored function *roomTemp* representing the sensed rooms

³<https://www.openhab.org/docs/concepts/>


```

-- specification 6 (setHeatingStatus(ch) != OFF => 0 (roomTempSoved(ch) + roomTempSoved(h1)) / 2 < 18) is true
-- specification 7 (setHeatingStatus(ch) != OFF => 0 (roomTempSoved(ch) + roomTempSoved(h1)) / 2 < 18) is true
-- specification 8 (F setHeatingStatus(ch) = FAIRLY_HOT) is false
-- specification 9 (F setHeatingStatus(ch) = VERY_HOT) is false
-- specification 10 (F setHeatingStatus(ch) = FAIRLY_HOT) is false
-- specification 11 (F setHeatingStatus(ch) = VERY_HOT) is false
-- specification 12 (F (roomTempSoved(ch) + roomTempSoved(h1)) / 2 < 18 => F (setHeatingStatus(ch) != OFF & setHeatingStatus(ch) != OFF)) is true

```

Figure 5. Example of verification of a MAPE loop in ASMETA

```

1 rule Exec_E_ch
2 when
3   Item start_E_ch received command ON
4 then
5   //Insert here the exec logic
6   sendCommand(start_E_ch, OFF) //trigger clean-up
7   sendCommand(start_E_hb, ON) //triggering of component E of hb
8   sendCommand(start_E_h1, ON) //triggering of component E of h1
9 end

```

Figure 6. Example of MAPE component for the OpenHAB rule engine

temperatures. Starting from State 1, the ASM evolves autonomously executing all the computations of the MAPE components, till reaching State 3 where the heaters of the rooms are turned on by setting to FAIRLY_HOT the thermostats (setHeatingStatus).

If the designer wants a higher confidence on the correctness of the designed loop, he/she can specify some temporal logic properties (e.g., “whenever necessary, the adaptation is performed”; “the adaptation is only performed when necessary”; “all possible kinds of adaptation can be executed”; etc.). These system-dependent properties can then be checked with the AsmetaSMV model checker of ASMETA. Fig. 5 shows the verification result of a set of properties for the CH adaptation concern in the verification environment of ASMETA⁴. Other verification techniques dealing with system-independent properties (or meta-properties) of distributed MAPE loops (e.g., non-conflict, minimality, etc.) may also be considered, as devised in [4].

The MAPE loop structure of an MSL model can be then coded automatically in the OpenHAB rule engine to leverage the MAPE functionality to do concrete home automation. Fig. 6 shows an example of OpenHAB rule template in the VS Code editor (an OpenHAB extension for the Visual Studio Code); it has been generated automatically for the component E of the main MAPE group (ch) for the running example. The rules definition (a file .rules) obtained from an MSL model can be then manually refined to complete the automation logic (the MAPE components behavior) by specifying OpenHAB-specific actions on the items⁵ declared in the managed home description in OpenHAB. The rules file can then be placed into the rules directory of OpenHAB. Rule files for different adaptation/control concerns (different MAPE-K loops) will be generated; they all share the same OpenHAB platform instance for a specific smart home (the managed subsystem and environment) and react on events on the OpenHAB event bus, i.e., commands for home devices.

⁴Note that properties 3-6 are expected to be false, as they return a behavior we are interested in observing.

⁵Items in OpenHAB represent capabilities of things (e.g., physical devices) in the managed home. They have a state and may receive commands.

IV. CONCLUSION

In this paper, we presented the MSL framework for defining and instantiating MAPE-K feedback loop patterns. A smart home automation example is used throughout the demo to illustrate the features and usage of the framework. To complement and complete the up-front design started in MSL, we presented two currently supported back-end frameworks: the ASMETA toolset, to perform V&V of MSL models, and the OpenHAB platform, to code and execute control automation processes of smart home gateways in terms of MAPE-K feedback loops. The MSL framework can be downloaded from the github project website [10]; the MSL model editor can be installed as Eclipse plugin⁶.

In the future, we want to extend MSL with composition strategies of patterns instances. Currently, MSL has been mostly validated in the home automation domain. More case studies targeting different domains are necessary to identify strengths as well as limitations of the approach. Using formal design and analysis methods (such as those provided by ASMs) has the advantage of increasing the quality of the software, but some expertise is required to use formal modeling and analysis tools, and to refine MAPE-K loop formal models into programmable modules. To this last purpose, we plan to investigate the applicability of some tools available in ASMETA to prove the correctness of refinement steps and allow model-to-code transformations. We also plan to support backward compatibility between an MSL model and its ASM counterpart.

Acknowledgment: We thank the UniBG student Federico Modica for helping us in developing the MSLtoOpenHAB generator.

REFERENCES

- [1] J. O. Kephart and D. M. Chess, “The vision of autonomic computing,” *IEEE Computer*, vol. 36, no. 1, pp. 41–50, 2003.
- [2] D. Weyns, B. R. Schmerl, V. Grassi, S. Malek, R. Mirandola, C. Prehofer, J. Wuttke, J. Andersson, H. Giese, and K. M. Göschka, *Software Engineering for Self-Adaptive Systems II: Int. Dagstuhl Seminar*. Springer, 2013, ch. On Patterns for Decentralized Control in Self-Adaptive Systems, pp. 76–107.
- [3] P. Arcaini, R. Mirandola, E. Riccobene, and P. Scandurra, “A DSL for MAPE patterns representation in self-adapting systems,” in *12th European Conference on Software Architecture, ECSA 2018, Springer LNCS 11048*, 2018, pp. 3–19.
- [4] P. Arcaini, E. Riccobene, and P. Scandurra, “Formal design and verification of self-adaptive systems with decentralized control,” *ACM Trans. Auton. Adapt. Syst.*, vol. 11, no. 4, pp. 25:1–25:35, Jan. 2017.
- [5] P. Arcaini, A. Gargantini, E. Riccobene, and P. Scandurra, “A model-driven process for engineering a toolset for a formal method,” *Softw., Pract. Exper.*, vol. 41, no. 2, pp. 155–166, 2011.
- [6] D. B. Abeywickrama, N. Hoch, and F. Zambonelli, “SimSOTA: Engineering and simulating feedback loops for self-adaptive systems,” in *Proc. of the Int. C* Conf. on Computer Science and Software Engineering*, ser. C3S2E '13. NY, USA: ACM, 2013, pp. 67–76.
- [7] F. Křikava, P. Collet, and R. B. France, “ACTRESS: Domain-specific Modeling of Self-adaptive Software Architectures,” in *Proc. of the 29th ACM Symposium on Applied Computing*, ser. SAC '14. NY, USA: ACM, 2014, pp. 391–398.
- [8] M. U. Iftikhar and D. Weyns, “Activforms: A runtime environment for architecture-based adaptation with guarantees,” in *2017 IEEE ICSA Workshops 2017, Gothenburg, Sweden*, 2017, pp. 278–281.
- [9] M. D’Angelo, A. Napolitano, and M. Caporuscio, “CyPhEF: A model-driven engineering framework for self-adaptive cyber-physical systems,” in *40th Int. Conf. on Software Engineering, ACM Proc.*, 2018, pp. 101–104.
- [10] “The MSL language,” <https://github.com/fmselab/msl>, 2018.

⁶The update site is <http://fmselab.di.unimi.it/sw/msl/updatesite/>